

Módulo 5 – Capítulo 02 – Sección 01

Anatomía de una llamada a la API: messages, system, role y contenido

Una llamada a la API de un LLM es, en esencia, un POST HTTP con un cuerpo JSON. Pero la semántica exacta de cada campo de ese JSON es el contrato sobre el que se construye toda la aplicación: si ese contrato se entiende superficialmente, los errores que emergen en producción son difíciles de diagnosticar porque ocurren en capas de abstracción muy alejadas del bug original. El campo que el desarrollador interpoló incorrectamente, el rol que está en el array equivocado, el límite de tokens que se ignora hasta que el modelo trunca una respuesta JSON a la mitad y rompe el parser: todos estos problemas tienen su raíz en una comprensión incompleta de la estructura del mensaje. Esta sección disecciona esa estructura con el nivel de detalle que la producción exige.

El campo `messages` es el núcleo de la llamada: un array ordenado de objetos donde el orden importa porque el modelo procesa los mensajes en secuencia y usa el historial para construir la respuesta. Cada objeto tiene al menos dos campos: `role` y `content`. El campo `role` define la perspectiva del mensaje dentro del diálogo — `system` para instrucciones globales de comportamiento, `user` para la entrada del usuario o del sistema externo que actúa como usuario, `assistant` para las respuestas previas del modelo que forman parte del historial conversacional. El campo `content` puede ser una cadena de texto simple para mensajes de texto puro, o un array de bloques de contenido para mensajes multimodales que mezclan texto, imágenes, audio y resultados de tool calls.

La semántica del rol `system` varía entre proveedores de forma que tiene consecuencias reales. En OpenAI, el mensaje de sistema es el primero del array `messages` con `role: "system"`; en Anthropic, el campo `system` vive fuera del array como campo independiente de primer nivel en el cuerpo del request, lo que habilita el prompt caching de ese contenido de forma independiente del historial conversacional. Esta diferencia estructural de Anthropic no es accidental: separar el system prompt del historial permite que las instrucciones estáticas del sistema —que raramente cambian entre turnos— se cacheen en el servidor y se reutilicen a

costo reducido en llamadas sucesivas de la misma sesión. En Google Gemini, el campo equivalente es `systemInstruction` con la estructura `{parts: [{text: "..."}]}`.

El rol `assistant` en el array `messages` sirve para dos propósitos. El primero es mantener el historial de una conversación multi-turno: cada respuesta del modelo se agrega al array antes del siguiente mensaje del usuario, permitiendo que el modelo tenga contexto de los turnos previos. El segundo —específico de Anthropic— es el "pre-fill" o prefillado de respuesta: colocar un mensaje `assistant` con un prefijo parcial como `{` o ````json` al final del array instruye al modelo a continuar desde ese prefijo, garantizando que la respuesta comience con ese texto. El pre-fill es la técnica más confiable para forzar salida JSON sin depender de instrucciones en el prompt que el modelo puede ignorar.

Conceptos clave de la estructura de mensajes

- **Campo `model`**: especifica el identificador exacto del modelo con versión (`claude-3-5-sonnet-20241022` , `gpt-4o-2024-08-06`), determinando capacidades, ventana de contexto y precio; nunca usar alias sin fecha que el proveedor puede actualizar silenciosamente.
- **Rol `system`**: establece el comportamiento, formato de salida, restricciones y persona del modelo; tiene mayor peso en el comportamiento que las instrucciones en mensajes `user`; en Anthropic vive fuera del array `messages` como campo de primer nivel.
- **Rol `assistant` para pre-fill**: en Anthropic, colocar `{"role": "assistant", "content": "{"}` como último elemento del array fuerza al modelo a comenzar la respuesta con `{`, garantizando salida JSON sin necesidad de parsear un JSON embebido en texto libre.
- **Bloques de contenido multimodal**: imagen en OpenAI como `{"type": "image_url", "image_url": {"url": "data:image/jpeg;base64,..."} }`; en Anthropic como `{"type": "image", "source": {"type": "base64", "media_type": "image/jpeg", "data": "..."} }`.
- **Campo `stop_sequences`**: array de strings que detienen la generación cuando el modelo los produce; útil para delimitar secciones de salida estructurada (`</output>` , `###`) o prevenir que el modelo continúe generando después de un delimitador conocido.

Nota del Arquitecto: El campo `max_tokens` es más crítico de lo que parece. Si el modelo está generando un JSON de respuesta y `max_tokens` se alcanza antes de que la respuesta esté completa, el JSON queda truncado e inválido —y el parser lanzará una excepción que puede enmascarse como un error del parser en lugar de un error de configuración. En sistemas donde la longitud de la respuesta es variable y

difícil de predecir, establecer `max_tokens` con suficiente margen (típicamente 2-3x la longitud media esperada) y monitorear el campo `finish_reason` en cada respuesta detecta los truncados antes de que rompan el parseo silenciosamente.

La estructura del array `messages` es el contrato principal entre la aplicación y el modelo. Comprenderla en detalle —no asumir que todos los proveedores la implementan de la misma forma— es el primer paso hacia una integración robusta. La sección siguiente explora los parámetros de inferencia: los controles sobre el proceso de muestreo que determinan cuán deterministas o variadas son las respuestas del modelo y qué impacto tienen en el costo y la latencia.

Principio rector: La estructura del array `messages` es el contrato principal entre la aplicación y el modelo; entender las diferencias semánticas entre proveedores —especialmente el campo `system` y el pre-fill de respuesta— y validar el array antes de cada llamada previene errores sutiles de contexto que son difíciles de diagnosticar en producción.

Módulo 5 – Capítulo 02 – Sección 02

Parámetros de inferencia: temperatura, top-p, top-k, max_tokens y su efecto

Los parámetros de inferencia son los controles sobre el proceso de muestreo del modelo: determinan cómo elige el siguiente token en cada paso de la generación, con consecuencias directas en la creatividad, la coherencia, la reproducibilidad, el costo y la latencia de las respuestas. La mayoría de los desarrolladores los configuran por convención — `temperature=0.7` para generación creativa, `temperature=0` para extracción estructurada— sin entender el mecanismo subyacente. Esa falta de comprensión lleva a configuraciones incorrectas que producen comportamientos sorprendentes: un sistema de extracción de entidades con `temperature=0.7` que devuelve valores ligeramente diferentes para el mismo input en cada llamada, o un sistema creativo con `temperature=0` que produce respuestas monótonamente repetitivas. Esta sección explica el mecanismo y los valores de producción recomendados para cada tipo de tarea.

La `temperature` escala los logits —las puntuaciones no normalizadas que el modelo asigna a cada token posible— antes de aplicar softmax para obtener probabilidades. A `temperature=0.0`, el modelo es efectivamente determinista: siempre elige el token con mayor probabilidad, produciendo la misma respuesta ante el mismo input (salvo por cambios en la infraestructura del servidor del proveedor). A `temperature=1.0`, los logits se aplican sin escala y el modelo muestrea de la distribución de probabilidad original. A `temperature > 1.0` (disponible solo en algunos proveedores), los logits se comprimen acercando las probabilidades entre sí, produciendo respuestas más diversas y menos predecibles. En Anthropic el rango es `0.0-1.0`; en OpenAI `0.0-2.0`.

El `top_p` (nucleus sampling) recorta el vocabulario al subconjunto de tokens cuya probabilidad acumulada alcanza el valor `p` antes de muestrear. Con `top_p=0.95`, el modelo elige entre los tokens que colectivamente representan el 95% de la masa de probabilidad, ignorando los tokens extremadamente improbables de la cola de distribución. Esto reduce la aparición de tokens absurdos sin hacer el modelo determinista. El `top_k` hace un recorte diferente: limita el vocabulario a los `k` tokens de mayor probabilidad sin importar su suma

acumulada. La recomendación de Anthropic es no combinar `top_p` y `top_k` con `temperature` simultáneamente, ya que sus efectos interactúan de forma compleja y raramente producen el comportamiento deseado; la práctica estándar es modificar `temperature` únicamente, y ajustar `top_p` solo si el comportamiento por defecto no es satisfactorio.

El parámetro `max_tokens` tiene dos efectos: limita la longitud de la respuesta generada y por ende el costo de tokens de salida, y determina el tiempo máximo de espera de la llamada (a mayor `max_tokens`, más tiempo puede tomar la generación). Un valor de `max_tokens` demasiado bajo produce respuestas truncadas —especialmente problemático para salidas JSON donde el truncamiento produce JSON inválido—; un valor demasiado alto genera costo innecesario si el modelo raramente produce respuestas largas. La práctica recomendada es monitorear la distribución de `output_tokens` en producción y establecer `max_tokens` en el percentil 99 de esa distribución más un margen del 20%.

Aspectos técnicos de los parámetros de muestreo

- `temperature=0.0` : produce respuestas casi deterministas, adecuado para extracción de entidades, clasificación, generación de código y cualquier tarea donde la reproducibilidad es requerida; temperatura cercana a 0 pero no exactamente 0 a veces estabiliza casos límite en ciertos modelos.
- `temperature=0.7-1.0` : rango para generación creativa, redacción de contenido y respuestas conversacionales donde se desea variedad sin perder coherencia; valores superiores a 1.0 producen diversidad extrema que raramente mejora la calidad.
- `max_tokens` : limita la longitud de salida y tiene impacto directo en el costo y la latencia; monitorear `finish_reason = "max_tokens"` en producción detecta truncados que pueden romper parsers de salida estructurada.
- `top_p=0.95` : valor de producción estándar que retiene el 95% de la masa de probabilidad, balanceando diversidad y coherencia; raramente necesita ajuste si `temperature` está bien configurado.
- `stop_sequences` : complementa `max_tokens` para terminar la generación ante un delimitador específico como `</output>` o `###`, garantizando salidas bien delimitadas sin desperdiciar tokens en contenido post-delimitador.

Nota del Arquitecto: Documentar los valores exactos de temperatura y demás parámetros de inferencia usados en producción junto con la versión del prompt es

tan crítico como versionar el prompt mismo. Un cambio de `temperature=0.7` a `temperature=0.3` en un sistema de generación de contenido puede cambiar el tono de las respuestas de forma tan significativa como una reescritura del system prompt, y si ese cambio no está registrado en el historial del sistema de versionado, diagnosticar una degradación de calidad semanas después se convierte en arqueología de configuración.

Los parámetros de inferencia son los controles de ajuste fino del comportamiento del modelo. La sección siguiente aborda una dimensión diferente de la llamada: el streaming, que cambia el modelo de request-response a un flujo continuo de tokens y tiene implicaciones profundas en la arquitectura del servidor, el manejo de errores y la experiencia de usuario.

Buena práctica: Documentar los valores de temperatura y demás hiperparámetros junto con el prompt versionado; cambios en estos valores pueden alterar el comportamiento del sistema tanto como cambios en el texto del prompt y deben tratarse con el mismo rigor de versionado y revisión.

Módulo 5 – Capítulo 02 – Sección 03

Streaming de respuestas: implementación y casos de uso

Cuando un usuario envía un mensaje a un asistente conversacional y ve la pantalla en blanco durante cinco segundos antes de que aparezca cualquier texto, la experiencia percibida es la de un sistema lento, aunque el tiempo total de generación sea idéntico al de un sistema que transmite tokens en tiempo real. Esta diferencia de percepción es la razón fundamental por la que el streaming es el modo de entrega estándar en interfaces conversacionales: no reduce el tiempo de generación total, sino que elimina la espera en blanco al entregar el primer token entre 200 y 800 milisegundos después de enviar el request.

El mecanismo técnico detrás del streaming es Server-Sent Events (SSE), un protocolo HTTP donde el servidor mantiene la conexión abierta y envía eventos delimitados por `data: {...}\n\n` a medida que los tokens se generan. Cuando el cliente activa `stream=True` en OpenAI, recibe un iterador de objetos `ChatCompletionChunk` donde cada chunk tiene un campo `choices[0].delta.content` con el fragmento de texto generado. En Anthropic, el patrón preferido es el context manager: `with client.messages.stream(...) as stream: for text in stream.text_stream:`, que proporciona un iterador de strings directamente, ocultando la complejidad de los eventos SSE subyacentes. En ambos casos, acumular el texto completo para procesamiento posterior requiere concatenar manualmente los fragmentos en un buffer.

Exponer streaming desde un servidor FastAPI al cliente web requiere menos código de lo que parece: `StreamingResponse` con `media_type="text/event-stream"` y un generador `async` que re-transmite los chunks del proveedor hacia el cliente. Esta arquitectura de proxy de streaming tiene una ventaja importante: el servidor puede aplicar transformaciones sobre los chunks antes de enviarlos (redactar PII, filtrar contenido, enriquecer con metadata) sin que el cliente perciba latencia adicional.

El streaming tiene implicaciones importantes en el manejo de errores que no existen en el modo no-streaming. Si el modelo está generando JSON y la conexión se interrumpe a mitad de la respuesta, el JSON parcial es inválido. Los sistemas que necesitan parsear JSON deben

acumular el stream completo antes de intentar el parseo, no parsearlo incrementalmente. De manera similar, los tool calls llegan como deltas de JSON que se van acumulando hasta que el evento `finish_reason: "tool_calls"` indica que el bloque está completo y puede parsearse. Este manejo diferente de las respuestas estructuradas en modo streaming vs modo completo es uno de los aspectos más frecuentes de error en la implementación inicial.

Los timeouts en conexiones streaming merecen configuración explícita. Un proxy inverso o un load balancer con timeout de lectura de 60 segundos cerrará silenciosamente una conexión SSE si no llega ningún byte en ese intervalo, lo que ocurre en respuestas largas. La solución es configurar `timeout=httpx.Timeout(connect=10, read=300)` en el cliente HTTP subyacente y, en el lado del servidor, enviar comentarios SSE keep-alive periódicos (`: keep-alive\n\n`) si la arquitectura de red lo requiere.

Aspectos técnicos del streaming

- **Implementación en Python (OpenAI):** iterar sobre el stream devuelve objetos `ChatCompletionChunk`; el texto está en `chunk.choices[0].delta.content`; acumular con `full_text += chunk.choices[0].delta.content or ""` construye la respuesta completa.
- **Context manager en Anthropic:** `with client.messages.stream(...) as stream:` expone `stream.text_stream` para texto puro y `stream.get_final_message()` para obtener el objeto `Message` completo con tokens y metadatos al finalizar.
- **FastAPI StreamingResponse:** generador async que yield-ea `f"data: {json.dumps({'text': chunk})}\n\n"` con `media_type="text/event-stream"` permite retransmitir el stream del proveedor al cliente web de forma proxied.
- **Tool calls en streaming:** acumular los deltas de JSON del tool call en un buffer hasta `finish_reason == "tool_calls"`, luego parsear el buffer completo; nunca parsear deltas parciales de JSON.
- **Casos donde no usar streaming:** pipelines batch donde la respuesta se procesa antes de continuar (extracción de JSON, clasificación, embeddings) no se benefician del streaming y añaden complejidad de manejo de buffer sin ventaja de UX.

Nota del Arquitecto: Los experimentos de A/B testing sobre interfaces conversacionales muestran consistentemente que la tasa de abandono sube entre el 15% y el 30% cuando se reemplaza streaming por respuesta completa, incluso manteniendo idéntico el tiempo total de generación. El usuario que ve texto aparecer progresivamente percibe que el sistema está trabajando; el usuario que ve una

pantalla en blanco durante el mismo intervalo percibe que el sistema no responde. Esta asimetría de percepción hace que el streaming sea una decisión de producto, no solo de implementación técnica.

En la sección siguiente abordaremos la gestión de errores: cómo manejar rate limits, timeouts y fallos del proveedor con estrategias de reintento que mantienen la resiliencia del sistema sin amplificar el problema ante degradaciones del servicio externo.

Idea central: El streaming no es solo una optimización de UX: en interfaces conversacionales elimina la espera en blanco que los usuarios interpretan como fallo del sistema, reduciendo la tasa de abandono y aumentando la percepción de calidad independientemente del tiempo total de generación.

Módulo 5 – Capítulo 02 – Sección 04

Gestión de errores: rate limiting, timeouts y reintentos con backoff exponencial

Las APIs de LLM son dependencias externas con disponibilidad imperfecta: el SLA típico del 99.5% de los proveedores principales equivale a hasta 44 horas de indisponibilidad acumulada por año, y las degradaciones parciales —donde el servicio responde pero con latencia elevada o tasa de error aumentada— ocurren con mayor frecuencia que las interrupciones completas. Un sistema de producción que no trata los errores transitorios como ciudadanos de primera clase convierte cada degradación del proveedor en una indisponibilidad del servicio propio, propagando hacia arriba un problema que podría haberse absorbido con una estrategia de reintento correctamente implementada.

Los errores que produce una API de LLM se dividen en dos categorías con estrategias de manejo radicalmente diferentes. Los errores retryables —429 (rate limit excedido), 500/502/503 (error interno del servidor), 529 (sobrecarga temporal, específico de Anthropic), y timeouts de conexión o lectura— son transitorios: el mismo request puede tener éxito si se reintenta después de un tiempo de espera. Los errores no retryables —400 (parámetros inválidos), 401 (API key ausente o revocada), 422 (contenido que viola políticas de uso)— son permanentes: reintentar el mismo request producirá el mismo error, por lo que el reintento automático solo desperdicia recursos y cuota. Confundir estas categorías —reintentar un 401 o no reintentar un 429— es un error de implementación frecuente con consecuencias operacionales.

La respuesta correcta a un error 429 no es reintentar inmediatamente sino esperar el tiempo indicado en el header `Retry-After` cuando está presente, o aplicar backoff exponencial con jitter cuando no lo está: `wait = min(cap, base * 2^n) + random.uniform(0, 1)`. El jitter aleatorio es esencial: sin él, múltiples workers que fallan simultáneamente ante el mismo rate limit reintentan al mismo tiempo y amplifican el problema en lugar de distribuirlo en el tiempo. La biblioteca `tenacity` en Python implementa este patrón con una API declarativa:

```
@retry(wait=wait_exponential(multiplier=1, min=4, max=60),
stop=stop_after_attempt(5), retry=retry_if_exception_type((RateLimitError,
```

`APIStatusError)))` . Los SDKs oficiales de OpenAI y Anthropic ya incluyen retry automático con backoff por defecto (2 reintentos), pero para sistemas de alto volumen o con requisitos de resiliencia específicos, configurar la estrategia con `tenacity` da control total sobre el número de intentos, el tipo de backoff y el logging de cada intento fallido.

Los timeouts deben configurarse en dos niveles independientes: el timeout de conexión (tiempo para establecer la conexión TCP, típicamente 10-30 segundos) y el timeout de lectura (tiempo para recibir la respuesta completa o el primer byte en streaming, típicamente 60-300 segundos dependiendo de la longitud máxima esperada de las respuestas). Usar un timeout único sin diferenciación entre conexión y lectura puede resultar en que respuestas largas sean truncadas por un timeout de lectura demasiado corto, o que el sistema espere indefinidamente si la conexión nunca se establece. `httpx.Timeout(connect=10.0, read=120.0, write=10.0, pool=5.0)` da granularidad completa sobre cada fase de la conexión HTTP.

Puntos críticos en gestión de errores de API

- **Backoff exponencial con jitter:** la fórmula `wait = min(cap, base * 2^n) + random(0, 1)` distribuye los reintentos en el tiempo; el parámetro `cap` previene esperas excesivas (ej. `cap=60` segundos); el jitter aleatorio evita el "thundering herd" cuando múltiples workers fallan simultáneamente.
- **Diferenciación por categoría de error:** errores 400, 401 y 422 se propagan inmediatamente como errores permanentes; errores 429, 500, 502, 503 y 529 se reintenten con backoff; timeouts se reintenten con backoff pero con menor número de intentos para no bloquear recursos por demasiado tiempo.
- **Circuit breaker para degradaciones prolongadas:** después de N fallos consecutivos en una ventana de tiempo, el circuit breaker abre y rechaza inmediatamente las llamadas durante un período de cooldown, devolviendo un error rápido o activando un fallback; el capítulo 10 desarrolla este patrón en detalle.
- **Dead letter queue en sistemas asíncronos:** mensajes que fallan tras el máximo de reintentos se enrutan a una DLQ (SQS, RabbitMQ) para inspección manual en lugar de perderse silenciosamente; es la red de seguridad para requests que ningún número de reintentos puede salvar.
- **Logging contextual de errores:** incluir en cada log de error el `request_id` del proveedor, el número de intento, el modelo usado, los tokens del prompt, y el tipo de error específico facilita el diagnóstico retrospectivo en sistemas de alta concurrencia.

La gestión de errores no es una funcionalidad que se añade después de que el sistema funciona: es una dimensión de diseño que se decide desde el primer sprint. El siguiente paso es comprender el costo de las llamadas al LLM y cómo gestionarlo activamente con conteo de tokens, caché de prompts y elección estratégica del modelo para cada tarea.

Para recordar: Un sistema que no maneja correctamente los errores transitorios de una API externa no es un sistema de producción; la resiliencia ante fallos del proveedor debe diseñarse desde el primer sprint, con distinción clara entre errores retryables y no retryables, backoff con jitter y timeouts explícitos en ambas fases de la conexión.

Módulo 5 – Capítulo 02 – Sección 05

Gestión de costos: conteo de tokens, caché de prompts y optimización de llamadas

El costo de una API de LLM no se paga por llamada sino por token, y la diferencia entre un sistema de IA con gasto controlado y uno con facturas sorpresa mensuales reside casi siempre en la misma causa: nadie instrumentó el conteo de tokens desde el inicio. Un sistema que envía prompts sin acotar su longitud puede acumular contextos conversacionales de 50.000 tokens en sesiones largas, elevando el costo de un request de \$0.006 a \$0.18 sin que ningún desarrollador lo note hasta que llega la factura del proveedor. La gestión activa del costo comienza por entender cómo se compone.

La factura de una llamada al LLM suma cuatro componentes. Los tokens de entrada —system prompt, historial conversacional, documentos recuperados por RAG, ejemplos few-shot, y la query del usuario— representan el mayor volumen de tokens pero el menor precio por token. Los tokens de salida —la respuesta generada— son 3 a 5 veces más caros por token que los de entrada en la mayoría de los proveedores: Claude 3.5 Sonnet factura \$3 por millón de tokens de entrada y \$15 por millón de tokens de salida. Los tokens de caché de lectura, disponibles en Anthropic con el mecanismo de prompt caching, cuestan \$0.30 por millón —un 90% de descuento sobre el precio normal de entrada— cuando el prefijo fue previamente escrito al caché. Los tokens de caché de escritura cuestan \$3.75 por millón, un 25% más caro que la lectura normal, pero el breakeven se alcanza con solo 1.25 llamadas que reutilicen el mismo prefijo.

El conteo previo de tokens antes de ejecutar una llamada costosa es la herramienta más simple y efectiva para controlar el gasto. `tiktoken.encoding_for_model("gpt-4o").encode(prompt)` calcula los tokens de un texto para modelos OpenAI en microsegundos, sin llamada a red. El SDK de Anthropic expone `client.messages.count_tokens(model=..., messages=...)` que devuelve el conteo exacto usando el mismo tokenizador que el modelo. Instrumentar este conteo en el código de construcción del prompt permite detectar en desarrollo cuando el contexto crece inesperadamente —por ejemplo, un bug que acumula el historial completo sin límite— antes de que ese bug llegue a producción.

El prompt caching de Anthropic representa el mayor salto de eficiencia económica disponible para sistemas con contenido estático repetitivo. Marcar el system prompt y los documentos de referencia con `cache_control: {"type": "ephemeral"}` hace que esos tokens se cacheen en los servidores de Anthropic por cinco minutos (renovables con cada uso). En un asistente corporativo donde el system prompt contiene 3.000 tokens de instrucciones y políticas de empresa, y donde los usuarios hacen múltiples preguntas en una sesión, el ahorro por sesión es del 87% en los tokens de entrada del system prompt. La condición para maximizar el hit rate es estructural: el contenido estático debe ir siempre al inicio del prompt —system prompt, documentos, ejemplos few-shot— y el contenido variable —la query del usuario, los últimos turnos del historial— debe ir al final.

Estrategias de optimización de costos

- **Conteo previo de tokens:** usar `tiktoken` (OpenAI) o `client.messages.count_tokens()` (Anthropic) antes de llamadas costosas para detectar prompts que exceden el presupuesto y alertar sobre contextos anormalmente grandes en desarrollo.
- **Prompt caching con `cache_control`:** colocar el contenido estático (instrucciones del sistema, documentos de referencia, ejemplos few-shot) al inicio del prompt con `cache_control: {"type": "ephemeral"}` y el contenido variable al final; el hit rate del caché depende directamente de esta estructura.
- **Elección de modelo por tarea:** usar Claude 3.5 Haiku o gpt-4o-mini para clasificación, extracción de entidades y routing, reservando Claude 3.5 Sonnet o gpt-4o para generación de contenido complejo; la diferencia de precio es de 10 a 20 veces para tareas donde la diferencia de calidad es de 2 a 5 puntos.
- **Batching de requests:** consolidar múltiples consultas independientes en una sola llamada usando separadores XML cuando la latencia no es crítica, reduciendo el overhead del system prompt repetido y el costo fijo por request.
- **Truncación inteligente del historial conversacional:** implementar un límite de tokens configurable (`MAX_CONTEXT_TOKENS=8000`) y comprimir el historial anterior con una llamada de resumen al propio LLM cuando se supera el umbral, manteniendo el contexto útil sin crecimiento ilimitado.

Nota del Arquitecto: En proyectos de mediana escala (50.000-200.000 requests por mes), la implementación de prompt caching en system prompts largos y la migración de tareas simples a modelos más económicos típicamente reduce el gasto

en un 40-60% sin ningún cambio perceptible en la calidad para el usuario. El retorno de invertir una semana de ingeniería en la optimización de costos es frecuentemente mayor que el retorno de cualquier otra mejora de esa semana. El prerequisite es tener el sistema de observabilidad que muestre el breakdown de costos por feature, por modelo y por tipo de token.

El costo de los tokens de salida domina el gasto en sistemas de generación intensiva; cada token de salida innecesario es evitable con prompts que instruyen al modelo a ser conciso y a usar JSON compacto en lugar de narrativa extendida cuando los datos son lo que importa, no la prosa que los rodea.

Principio rector: El costo total de un sistema de IA es predecible y gestionable cuando cada llamada se instrumenta con el conteo exacto de tokens de entrada, salida y caché; la variabilidad de costo sin instrumentación convierte el presupuesto mensual en una sorpresa en lugar de una métrica bajo control.

Módulo 5 – Capítulo 02 – Sección 06

Cierre: buenas prácticas para consumir APIs de LLM en producción

Llevar un sistema que consume APIs de LLM desde un prototipo funcional hasta un servicio de producción confiable requiere más que hacer que el "camino feliz" funcione correctamente. Las APIs de LLM son dependencias externas con tres características que las distinguen de cualquier otra dependencia de un sistema web: latencia variable que puede ir de 50 milisegundos a 30 segundos en la misma aplicación; costo por uso que puede dispararse ante errores de programación como loops sin límite o prompts sin control de longitud; y comportamiento probabilístico donde la misma entrada puede producir salidas diferentes. Cada una de estas características requiere disciplinas de ingeniería específicas que no se necesitan cuando la dependencia es una base de datos relacional con latencia de 5 milisegundos y comportamiento determinista.

La primera disciplina es la resiliencia. Todo el manejo de errores desarrollado en la sección anterior —retry con backoff exponencial, diferenciación entre errores retryables y permanentes, timeouts explícitos en conexión y lectura— debe estar encapsulado en una capa de integración única, no disperso en cada punto de llamada de la aplicación. La forma más simple de implementar esto es un wrapper de cliente: una función o clase `LLMClient` con un método `complete()` que internamente aplica el retry, el logging y el manejo de errores, y que todos los módulos de la aplicación usan en lugar del SDK directamente. Esta centralización tiene tres beneficios: los bugs de resiliencia se corrigen en un solo lugar, el logging es consistente en toda la aplicación, y el cambio de proveedor requiere modificar solo el wrapper.

La segunda disciplina es la gestión activa del costo. Además de las estrategias de la sección anterior, los límites de gasto configurables en la consola del proveedor son la red de seguridad contra escenarios catastróficos: OpenAI y Anthropic permiten configurar alertas por email y por webhook cuando el gasto mensual supera umbrales definidos (50%, 80%, 100% del presupuesto). Esta protección es especialmente importante en los primeros meses de un proyecto, cuando los bugs de contexto o los loops inadvertidos pueden generar cargos

significativos antes de ser detectados. El uso de API keys separadas por ambiente — desarrollo, staging y producción con límites de gasto diferenciados— evita que los experimentos de desarrollo consuman la cuota de producción.

La tercera disciplina es la trazabilidad. Versionar los prompts como código —almacenados en archivos en el repositorio con identificadores de versión, revisados en PRs, con un changelog que documenta qué cambió y por qué— es tan importante como versionar el código de aplicación. El comportamiento de un sistema de IA está determinado en igual medida por el código Python que lo orquesta y por los prompts que guían al modelo; un cambio de prompt sin versionar es un cambio de comportamiento sin trazabilidad, el tipo de cambio que produce los incidentes más difíciles de diagnosticar en producción porque el sistema sigue respondiendo sin errores de runtime mientras la calidad se degrada silenciosamente.

La cuarta disciplina es el testing sin costo real. Las llamadas al LLM en tests unitarios deben reemplazarse por mocks del cliente que devuelven respuestas predefinidas: `mock.patch.object(client, 'messages')` con un valor de retorno configurado al objeto Message que el test necesita. Esta práctica hace los tests rápidos, gratuitos, reproducibles y ejecutables en cualquier entorno sin credenciales de API. Las llamadas reales al LLM se reservan para los tests de integración y de evaluación de calidad, que se ejecutan bajo demanda y no en cada commit.

Buenas prácticas consolidadas

- **Wrapper de cliente con observabilidad centralizada:** encapsular el SDK del proveedor en una función o clase que añade logging, métricas y manejo de errores; toda la aplicación usa este wrapper, no el SDK directamente.
- **Validación de salida estructurada:** si se espera JSON, validar con Pydantic o `json.loads()` dentro de un try/except e implementar al menos un reintento con instrucción de corrección antes de propagar el error al llamante.
- **Límites de gasto configurables:** configurar alertas en la consola del proveedor al 50%, 80% y 100% del presupuesto mensual, y usar API keys separadas con límites diferenciados por ambiente.
- **Separación de ambientes:** claves distintas para desarrollo, staging y producción, con límites de gasto que reflejan el volumen esperado de cada ambiente y evitan contaminación de cuota.
- **Testing sin costo real:** mocks del cliente LLM en tests unitarios para toda la lógica de Python; llamadas reales reservadas para tests de evaluación de calidad ejecutados bajo

demanda en pipelines de CI con triggers condicionales.

La simplicidad en el código de integración —funciones pequeñas con responsabilidad única, validación explícita de entrada y salida, manejo deliberado de errores— no es una cuestión estética sino el predictor más confiable de que el sistema podrá diagnosticarse y mantenerse cuando sucedan los problemas que inevitablemente suceden en producción. El capítulo siguiente aborda la siguiente capa del stack: los frameworks de orquestación que emergen cuando los flujos de múltiples pasos justifican una abstracción adicional sobre el SDK puro.

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand." — Martin Fowler. En AI Engineering, esto se traduce en que el código de integración con APIs de LLM debe ser tan legible y mantenible como cualquier otro código de producción, independientemente de la complejidad del modelo que orquesta.